



Department of
Computer Science and Engineering
University of Dhaka

**Assignment 1: ARM-Based Healthcare
Monitoring & Billing System**

CSE 2106:
Microprocessor and Assembly Language Lab
2nd Year 1st Semester 2025

Date of submission: 15 December 2025

Group No: 14

Submitted to:

Dr. Mosarrat Jahan, Associate Professor, CSEDU
Palash Roy, Lecturer, CSEDU

Submitted by:

Istiaq Mahmud – 06
Ahmed Reza Tausif – 12
Mayesha Binte Liton – 62

SmartCare-32: ARM-Based Healthcare Monitoring System

CSE 2106: Microprocessor and Assembly Language Lab

Group: 14

15 December 2025

Contents

1	Introduction	3
2	File Inventory	3
3	Module Descriptions & Simulation Analysis	4
3.1	Module 1: Patient Record Initialization	4
3.2	Module 2: Vital Sign Data Acquisition	4
3.3	Module 3: Vital Threshold Alert Module	4
3.4	Module 4: Medicine Administration Scheduler	5
3.5	Module 5: Treatment Cost Computation	5
3.6	Module 6: Daily Room Rent Calculation	5
3.7	Module 7: Medicine Billing Module	6
3.8	Module 8: Bill Aggregator	6
3.9	Module 9: Sorting Patients by Criticality	6
3.10	Module 10: UART Summary Report Generator	7
3.11	Module 11: System Error Detection & Logging	7
4	UART/ITM Output Layer	7
5	Quick Reference: Consolidated Outputs	8
6	Debug (printf) Viewer Output	9
7	Output Analysis	10
8	Final Results for All Patients	11
8.1	1. Patient 2 — (Most Critical)	11
8.2	2. Patient 3 — (Moderate)	12
8.3	3. Patient 1 — (Stable)	13
9	Quick Memory Verifications	13
10	Notes: Known Implementation Observations	14

1 Introduction

SmartCare-32 is a modular embedded firmware designed for the ARM Cortex-M architecture. The system simulates a hospital environment by managing patient records in RAM, acquiring vital signs from simulated sensors, and performing real-time billing computations.

The system demonstrates advanced Assembly concepts including:

- **Modular Design:** 13 separate files linked via `EXPORT/IMPORT`.
- **Memory Management:** Manual structure alignment and circular buffers.
- **Algorithms:** Bubble Sort for triage and Fixed-Point Arithmetic for billing.
- **Safety:** Real-time anomaly detection for hardware and software failures.

2 File Inventory

File	Role
<code>Data.s</code>	Defines global memory layout, patient structures, circular buffers, and lookup tables. Allocates RAM at <code>0x20000000</code> .
<code>Main.s</code>	The central orchestrator. Initializes the system and runs the infinite monitoring loop, calling all modules sequentially.
<code>Mod1_Init.s</code>	Initializes 3 patient records in RAM with hardcoded IDs (<code>0x1111...</code>) and test data.
<code>Mod2_Acquisition.s</code>	Simulates sensor reading and manages the 10-entry circular buffer for HR, BP, and O2.
<code>Mod3_Alerts.s</code>	Checks vital signs against safety thresholds (e.g., HR $\geq$ 120) and logs alerts if violated.
<code>Mod4_Scheduler.s</code>	Monitors the system clock to determine if medication dosage is due.
<code>Mod5_Treatment.s</code>	Computes treatment costs using a Lookup Table (LUT) for $O(1)$ efficiency.
<code>Mod6_Room.s</code>	Calculates room rent using fixed-point arithmetic to apply a 5% discount for long stays.
<code>Mod7_MedBill.s</code>	Iterates through a linked medicine list to calculate total medication costs.
<code>Mod8_Aggregator.s</code>	Sums all costs into a Total Bill, using <code>ADDS</code> and <code>BVS</code> to protect against integer overflow.
<code>Mod9_Sort.s</code>	Implements Bubble Sort to reorder patient structures in RAM based on Criticality (Alert Count).
<code>Mod10_UART.s</code>	Converts binary data (ID, Bill) to ASCII and transmits it via the UART Data Register.
<code>Mod11_Anomaly.s</code>	"Watchdog" module: detects stuck sensors and memory boundary overflows.

Table 1: Project File Structure

3 Module Descriptions & Simulation Analysis

3.1 Module 1: Patient Record Initialization

What it does: Populates the RAM block starting at 0x20000000 with initial patient data, setting up IDs, demographics, and billing rates.

Solution Technique: We used **Structure Padding** to ensure strict 32-bit alignment. A 1-byte padding is inserted after the 8-bit **Age** field so that the 16-bit **Ward** field aligns perfectly to offset 0x0A.

Core ARM Assembly Structure:

```
1 LDR R0, =Patient_Base_Addr ; 0x20000000
2 LDR R1, =0x11111111         ; Patient ID
3 STR R1, [R0, #0]           ; Store at Offset 0
```

Output (Memory State):

- **Patient 1 (Base):** ID=0x11111111, Age=25, Alerts=2.
- **Patient 2 (+28 bytes):** ID=0x22222222, Age=50, Alerts=10.
- **Patient 3 (+56 bytes):** ID=0x33333333, Age=40, Alerts=5.

3.2 Module 2: Vital Sign Data Acquisition

What it does: Continuous acquisition of simulated sensor data (HR, BP, O2) into a rolling circular buffer.

Solution Technique: Modulo-10 Arithmetic. The index increments from 0 to 9. Upon reaching 10, the logic `CMP R5, #10` resets it to 0. This creates a "Ring Buffer" that never overflows memory.

Core ARM Assembly Structure:

```
1 LSL R6, R5, #2 ; Offset = Index * 4
2 STR R0, [R3, R6] ; Store into Buffer
3 ADD R5, R5, #1 ; Increment
4 CMP R5, #10 ; Check Wrap-around
5 MOVEQ R5, #0 ; Reset if 10
```

Output:

- **Buffer State:** The `HR_Buffer` at 0x20000060 contains 10 consecutive readings.
- **Index:** Cycles $0 \rightarrow 9 \rightarrow 0$.

3.3 Module 3: Vital Threshold Alert Module

What it does: Compares the latest vital reading against safety limits (HR $\geq$ 120, O2 $\leq$ 92, BP $\geq$ 160/90).

Solution Technique: We use conditional branching (`BLE`, `BGE`). If a threshold is violated, a 16-byte record is written to the `Alert_Buffer` and the patient's `Alert_Count` is incremented.

Core ARM Assembly Structure:

```

1  CMP R0, #120           ; Check Heart Rate
2  BLE Check_02           ; If HR <= 120, skip
3  MOV R3, #1             ; Alert Type 1 (HR High)
4  BL Log_Alert           ; Call logging subroutine

```

Output (Simulation):

- **Scenario:** HR input set to 130bpm.
- **Result:** Alert_Count increments from 2 to 3.
- **Log:** New record created at Alert_Buffer with Type 0x01.

3.4 Module 4: Medicine Administration Scheduler

What it does: Checks if the current system time exceeds the Next_Due_Time for any medication.

Solution Technique: We calculate $Next_Due = Last_Admin + Interval$. The system compares $Current_Time \geq Next_Due$ to set a flag.

Output:

- **Current Time:** 10 (Simulation Clock).
- **Status:** No medicine overdue (Flag = 0).

3.5 Module 5: Treatment Cost Computation

What it does: Fetches the treatment code and retrieves the cost from a Lookup Table.

Solution Technique: Direct Indexing. We calculate the memory address as $Base_Address + (Code \times 4)$. This provides $O(1)$ efficiency.

Core ARM Assembly Structure:

```

1  LDR R2, =Costs_LUT
2  LSL R1, R1, #2         ; Index * 4 (Word alignment)
3  LDR R3, [R2, R1]       ; Fetch Cost from Table

```

Output (Patient 1):

- **Input:** Treatment Code 2.
- **Lookup:** Index 2 in table corresponds to 1,500.
- **Result:** Treatment_Cost = 1,500.

3.6 Module 6: Daily Room Rent Calculation

What it does: Computes $Rate \times Days$. If $Days > 10$, it applies a 5% discount.

Solution Technique: Fixed-Point Arithmetic. To avoid floating-point overhead, we calculate the discount as $(Total \times 95)/100$.

Core ARM Assembly Structure:

```

1  CMP R2, #10           ; Check if days > 10
2  BLE Save_Rent         ; Skip if not
3  MOV R4, #95
4  MUL R3, R3, R4        ; Total * 95
5  MOV R4, #100
6  UDIV R3, R3, R4       ; Result / 100

```

Output (Patient 1):

- **Raw:** $2,000 \text{ rate} \times 12 \text{ days} = 24,000$.
- **Discount:** $24,000 \times 95 / 100$.
- **Result:** `Room_Cost` = **22,800**.

3.7 Module 7: Medicine Billing Module

What it does: Iterates through a linked list of medicines to sum up the total cost.

Output:

- **Item A:** $10 \times 3 \times 5 = 150$.
- **Item B:** $50 \times 2 \times 1 = 100$.
- **Result:** `Medicine_Cost` = **250**.

3.8 Module 8: Bill Aggregator

What it does: Sums Treatment + Room + Medicine.

Solution Technique: Overflow Safety. We use the ADDS instruction (Add and Set Flags). If the **V (Overflow)** flag is set, we branch to an error handler that caps the bill at `MAX_INT` (`0x7FFFFFFF`).

Output (Patient 1):

- **Sum:** $22,800 + 1,500 + 250$.
- **Final Bill:** **24,550** (`0x5FE6`).
- **Status:** Overflow Flag = 0 (Safe).

3.9 Module 9: Sorting Patients by Criticality

What it does: Reorders patients in memory so the most critical (Highest Alert Count) is at the top.

Solution Technique: Bubble Sort. We compare the `Alert_Count` at offset `0x18`. If $Patient_A < Patient_B$, we swap the *entire* 28-byte structure using a byte-copy loop.

Output (Post-Sort Memory Order):

1. **Rank 1:** Patient 2 (Alerts: 10) → Address `0x20000000`.
2. **Rank 2:** Patient 3 (Alerts: 5) → Address `0x2000001C`.
3. **Rank 3:** Patient 1 (Alerts: 2) → Address `0x20000038`.

3.10 Module 10: UART Summary Report Generator

What it does: Converts binary data (ID, Bill) to ASCII and transmits it via the UART Data Register.

Output (Serial Terminal):

```
ID: 22222222 | Bill: 10500 | Status: CRITICAL
ID: 33333333 | Bill: 15500 | Status: MODERATE
ID: 11111111 | Bill: 24550 | Status: STABLE
```

3.11 Module 11: System Error Detection & Logging

This module functions as a "Watchdog," performing three distinct safety checks during every system cycle.

- **11a) Sensor Malfunction:** Maintains sensor_history arrays for HR, O2, SBP, DBP. If all 10 samples for a sensor are identical, it treats it as a "stuck sensor". Logic checks HR first. If HR is stuck, it logs **SENSOR MALFUNCTION** (Code 0x01) and returns.
- **11b) Invalid Dosage:** Validates medicine data. Iterates through the list; if `unit_price == 0`, logs **INVALID DOSAGE** (Code 0x03).
- **11c) Memory Overflow:** Checks if the patient pointer exceeds **RAM_LIMIT** (0x20002000) or if the total bill exceeds 0xF0000000. If so, logs **MEMORY OVERFLOW**.

4 UART/ITM Output Layer

What it does

Provides simulated serial printing using ARM ITM registers or direct UART writes:

- **ITM_Init:** Enables ITM trace control and stimulus port 0.
- **ITM_SendChar:** Polls the register to wait for the port to be ready, then writes the character.

Output Role

All printed reports (Module 10 and Module 11 print routines) go out through **ITM_SendChar**, allowing the Keil Simulator to display the output in the Debug (printf) Viewer console.

5 Quick Reference: Consolidated Outputs

- **Module 1:** Patient fields populated at 0x20000000, buffers cleared.
- **Module 2:** Buffer wraps at index 10. Address 0x20000060.
- **Module 3 (Alerts):**
 - P1: 2 $\rightarrow$ 2
 - P2: 10 $\rightarrow$ 11 (Critical)
 - P3: 5 $\rightarrow$ 5
- **Module 4:** Dosage Flag = 0 (Time $\neq$ Interval).
- **Module 5 (Treatment):** P1: 1,500 — P2: 100 — P3: 500.
- **Module 6 (Room):** P1: 22,800 — P2: 10,000 — P3: 15,000.
- **Module 7 (Medicine):** P1: 250 — P2: 400 — P3: 0.
- **Module 8 (Total Bill):**
 - P1: 53,900
 - P2: 122,200
 - P3: 50,500
- **Module 9 (Sorted):** P2 (Critical) $\rightarrow$ P3 (Moderate) $\rightarrow$ P1 (Stable).

6 Debug (printf) Viewer Output

```
=====
                        SYSTEM ERROR LOG
=====
Total Errors: 1

-----
Error #1
  Type:      SENSOR MALFUNCTION (Code 0x01)
  Sensor:    Heart Rate
  Stuck Val: 80
  Timestamp: 10 sec
-----

PATIENT SUMMARY REPORT
-----
ID: 22222222
  Status:    CRITICAL (Alerts: 11)
  Total Bill: $ 10500
-----
ID: 33333333
  Status:    MODERATE (Alerts: 5)
  Total Bill: $ 15500
-----
ID: 11111111
  Status:    STABLE (Alerts: 2)
  Total Bill: $ 24550
-----
End of Report
```

Figure 1: Simulated UART Terminal Output

7 Output Analysis

Initial Order (Initialized in data.s)

Pos	Address	ID	Name	Alerts
0	0x20000000	1001	Patient 1	2 (0x02)
1	0x2000001C	1002	Patient 2	10 (0x0A)
2	0x20000038	1003	Patient 3	5 (0x05)

Final Sorted Order (By Criticality)

Pos	ID	Name	Alerts	Status	Bill
0	1002	Patient 2	11 (0x0B)	CRITICAL	10,500
1	1003	Patient 3	5 (0x05)	MODERATE	15,500
2	1001	Patient 1	2 (0x02)	STABLE	24,550

8 Final Results for All Patients

8.1 1. Patient 2 — (Most Critical)

Condition: High Vital Spike (HR $\geq$ 130).

Base Address: 0x20000000

Memory Dump (Base):

```
0x20000000:  22 22 22 22 00 00 00 00 32 00 66 00 00 00 00 00
0x20000010:  88 13 00 00 D4 07 00 20 0B 00 00 00 00 00 00 00
```

Basic Information

Offset	Field	Decimal	Hex
+0x00	patient_id	572662306	0x22222222
+0x04	name_ptr	0	0x00000000
+0x08	age	50	0x32
+0x0A	ward_number	102	0x0066
+0x0C	treatment_code	0	0x00
+0x10	room_daily_rate	5000	0x1388
+0x18	alert_count	11	0x0B

Vital Signs (Simulated Capture)

Source	Field	Decimal	Hex
Register R0	heart_rate	130	0x82
Register R1	systolic_bp	165	0xA5
Register R2	oxygen_level	98	0x62

System Flags

Offset	Field	Decimal	Hex
(Global)	vital_index	1	0x01
(Global)	alert_flag	1	0x01
(Global)	dosage_due_flag	0	0x00

Billing

Offset	Field	Decimal	Hex
(Global)	treatment_cost	100	0x00000064
(Global)	room_cost	10,000	0x00002710
(Global)	total_bill	10,500	0x00002904

8.2 2. Patient 3 — (Moderate)

Condition: Elevated Vitals (Alerts = 5).

Base Address: 0x2000001C

Memory Dump (Base):

```
0x2000001C: 33 33 33 33 00 00 00 00 28 00 67 00 01 00 00 00
0x2000002C: B8 0B 00 00 D4 07 00 20 05 00 00 00 00 00 00 00
```

Basic Information

Offset	Field	Decimal	Hex
+0x00	patient_id	858993459	0x33333333
+0x08	age	40	0x28
+0x0A	ward_number	103	0x0067
+0x0C	treatment_code	1	0x01
+0x10	room_daily_rate	3000	0x0BB8
+0x18	alert_count	5	0x05

Vital Signs (Simulated)

Source	Field	Decimal	Hex
Register R0	heart_rate	125	0x7D
Register R1	systolic_bp	130	0x82
Register R2	oxygen_level	96	0x60

System Flags

Offset	Field	Decimal	Hex
(Global)	vital_index	1	0x01
(Global)	alert_flag	1	0x01
(Global)	dosage_due_flag	0	0x00

Billing

Offset	Field	Decimal	Hex
(Global)	treatment_cost	500	0x000001F4
(Global)	room_cost	15,000	0x00003A98
(Global)	total_bill	15,500	0x00003C8C

8.3 3. Patient 1 — (Stable)

Condition: Normal Vitals.

Base Address: 0x20000038

Memory Dump (Base):

```
0x20000038:  11 11 11 11 00 00 00 00 19 00 65 00 02 00 00 00
0x20000048:  D0 07 00 00 D4 07 00 20 02 00 00 00 00 00 00 00
```

Basic Information

Offset	Field	Decimal	Hex
+0x00	patient_id	286331153	0x11111111
+0x08	age	25	0x19
+0x0A	ward_number	101	0x0065
+0x0C	treatment_code	2	0x02
+0x10	room_daily_rate	2000	0x07D0
+0x18	alert_count	2	0x02

Vital Signs (Simulated)

Source	Field	Decimal	Hex
Register R0	heart_rate	80	0x50
Register R1	systolic_bp	120	0x78
Register R2	oxygen_level	98	0x62

System Flags

Offset	Field	Decimal	Hex
(Global)	vital_index	0	0x00
(Global)	alert_flag	0	0x00
(Global)	dosage_due_flag	0	0x00

Billing

Offset	Field	Decimal	Hex
(Global)	treatment_cost	1,500	0x000005DC
(Global)	room_cost	22,800	0x00005910
(Global)	total_bill	24,550	0x00005FE6

9 Quick Memory Verifications

- **Sorted IDs:**
 - 0x20000000 → 22 22 22 22 (Patient 2)
 - 0x2000001C → 33 33 33 33 (Patient 3)
 - 0x20000038 → 11 11 11 11 (Patient 1)
- **Alert Counts (Offset +0x18):**

- 0x20000018 → 0B (11 Alerts)
- 0x20000034 → 05 (5 Alerts)
- 0x20000050 → 02 (2 Alerts)

- **Total Bills:**

- Patient 2 → 0x2904 (10,500)
- Patient 3 → 0x3C8C (15,500)
- Patient 1 → 0x5FE6 (24,550)

10 Notes: Known Implementation Observations

1. **Padding for Alignment:** Unlike standard packed structures, we implemented manual padding (1 byte after Age) to ensure the 16-bit Ward Number aligns to an even address (0x0A), optimizing bus access.
2. **Global Billing Storage:** Billing computation utilizes global accumulator variables rather than storing detailed billing history inside the patient struct to save RAM.
3. **Discount Arithmetic:** Room rent discounts utilize fixed-point math (multiply by 95, divide by 100) to avoid floating-point overhead.
4. **Sensor Malfunction Logic:** The check routine triggers on the first sensor it finds stuck (HR first), so it may not log BP or O2 issues if HR is already flagged.

11 Limitations

Simulation Only

- UART via ITM (not real UART)
- Flash writes simulated in RAM

Fixed Patient Count

- Maximum 3 patients
- Fixed medicine list size

Error Log Capacity

- 20 records
- Circular overwrite